

Why Ontologies Are the Missing Layer in AI Agent Architecture —

7 min read · Feb 5, 2026



Sai

Follow



Listen



Share



More

You've built the RAG pipeline and the multi-agent system. So why do your agents still fall

Open in app ↗

≡ Medium



retrieve information, call tools, and chain reasoning steps together. But when you drop them into a genuinely complex domain — healthcare, finance, enterprise software — they start hallucinating relationships, confusing categories, and making moves that reveal a fundamental gap: they have no *structural understanding* of the world they operate in.

That structural understanding has a name. It's called an **ontology**. And the technique for building ontologies that scale from universal truths to domain-specific details is called **ontological stratification** — or layered ontology.

Full disclosure: I'm writing this series as a practitioner learning in public. My background is in AI consulting — I've built RAG pipelines, multi-agent systems, and production agent architectures. But I kept encountering a structural gap that retrieval and prompt engineering alone couldn't fill. This series documents my deep dive into ontologies and knowledge graphs as the missing cognitive layer for agentic AI. It's part tutorial, part field notes, part thinking-out-loud. If you're an AI builder sensing the same gap, consider this an invitation to learn alongside me.

This is Part 1 of a 10-part series called *The Layered Mind*, where we'll go from zero to building layered knowledge graphs that serve as the cognitive backbone of AI agent systems.

What Is an Ontology (In Practical Terms)?

The word comes from philosophy — literally “the study of what exists.” But in computer science and AI, it has a precise, practical definition:

An ontology is a formal, explicit specification of a shared conceptualisation.

Every word carries weight.

Formal means machine-readable — not just a document that humans nod along to. **Explicit** means the concepts and their relationships are directly stated, not left for inference. **Shared** means multiple systems, agents, or teams can use the same ontology as common ground. **Conceptualisation** means it's a deliberate model of some slice of reality.

The simplest possible ontology might look like this in pseudo-notation:

```
...  
CONCEPTS: Person, Organization, Employment  
RELATIONS: Person - worksAt → Organization  
CONSTRAINTS: Employment has startDate, endDate, role  
A Person may worksAt multiple Organizations  
...
```

No code. No databases. Just a precise, shared agreement about what exists in this tiny world and how things connect.

If you've worked with database schemas, this will feel familiar — but ontologies go further. A schema defines storage structure. An ontology defines *meaning*. It captures not just “these fields exist” but “here is how these concepts relate to each other in the real world.”

Knowledge Graphs: Where Ontologies Become Data

An ontology is the blueprint. A **knowledge graph** is the building filled with inhabitants.

The ontology says: *"Persons can work at Organizations."*

The knowledge graph says: *"Ada Lovelace worked at the University of London."*

The atomic unit is the **triple**— ``Subject → Predicate → Object``:

```
```\n(Ada Lovelace) - worksAt → (University of London)\n(Ada Lovelace) - hasRole → (Mathematician)\n(University of London) - isA → (Organization)\n```
```

Each triple is a single fact. Chain enough of them together and you get a graph — a web of interconnected knowledge that machines can traverse, query, and reason over.

If you've built RAG systems with vector databases, here's the key distinction: **vector similarity tells you *what's related***; a **knowledge graph tells you *how and why things are related***. A vector store can surface that "Ada Lovelace" and "mathematics" are close in embedding space. A knowledge graph can tell you that she held a specific role, at a specific institution, during a specific period — and that the institution is an Organization located in a particular city.

That structural precision is what agents need for real reasoning.

## Why Flat Knowledge Breaks Down ?

Most teams who experiment with knowledge graphs build one big, flat ontology for their domain, populate it with data, and connect it to their agent. It works initially. Then three problems emerge.

**Every domain reinvents basic concepts.** Your healthcare system defines “Person” one way. Your finance system defines it another. Your personal assistant has yet another implicit definition. There is no shared foundation, so integration is painful and brittle.

**Abstraction levels collapse together.** The concept “Event” is universal — it applies across all domains. “Medical Appointment” is healthcare-specific. “Schedule a follow-up with Dr. Chen next Tuesday” is a concrete, actionable instance. When these all live at the same ontological level, agents can’t distinguish between reasoning about *what kinds of things exist* versus *what to actually do right now*.

**Knowledge can’t transfer across agents.** If your coding agent develops structured knowledge about “Dependencies” and “Version Conflicts,” could your project management agent leverage that structural understanding? In a flat ontology, no. The concepts are locked into domain-specific silos with no shared abstractions to bridge them.

## Ontological Stratification: The Layer Cake

Ontological stratification solves these problems by organizing knowledge into **distinct layers**, each building on the one below:

...

Layer 4: APPLICATION / INSTANCE

"Schedule meeting with Dr. Chen Tue"

Layer 3: TASK ONTOLOGY

Scheduling, Diagnosing, Coding...

Layer 2: DOMAIN ONTOLOGY

Healthcare, Finance, Software Dev...

Layer 1: UPPER / FOUNDATIONAL ONTOLOGY

Entity, Event, Agent, Relation...

...

**Layer 1 — The Upper Ontology** defines concepts so general they apply everywhere: Entity, Event, Agent, Relation, Quality, Process. This is the universal language all your agents share. Well-known upper ontologies include BFO (Basic Formal Ontology), DOLCE, and SUMO.

**Layer 2 — Domain Ontologies** specialize those universal concepts for specific fields. In healthcare: Patient is-a Person, Diagnosis is-a Event, Symptom is-a Quality. In software development: Repository is-a Artifact, Bug is-a Event, Dependency is-a Relation.

**Layer 3 — Task Ontologies** model what agents *\*do\**: actions, goals, plans, capabilities. “Scheduling” is a task that operates on Events and Agents. “Diagnosing” is a task that connects Symptoms to Conditions. Each task knows which domain concepts it touches.

**Layer 4 — Application/Instance Layer** holds the actual data: real appointments, real patients, real code commits, real emails.

The power of this architecture is that **layers communicate through inheritance and specialization**. Your personal agent and your healthcare agent both understand “Event” because they share Layer 1. But they specialize it differently at Layer 2. And a new domain agent can be bootstrapped by plugging a new Layer 2 module onto the existing Layer 1 foundation.

## Why This Matters Now for Agentic AI

Three converging trends make layered ontologies newly urgent.

**Agents need structured memory.** Vector databases excel at semantic similarity, but agents making decisions need to traverse *constraints and relationships*. “Can I schedule this meeting?” is a constraint-satisfaction problem, not a nearest-neighbor search. Knowledge graphs provide the structured memory that makes this possible.

**Multi-agent systems need a shared language.** When a research agent, a planning agent, and an execution agent each maintain their own implicit model of the world, coordination collapses at the seams. A shared upper ontology gives them a formal lingua franca — a common foundation that each agent specializes for its own role.

**Domain specialization must be reusable.** You don't want to engineer knowledge from scratch every time your agent enters a new domain. Layered ontologies let you snap in new domain modules on top of a stable, tested foundation. The upper ontology is write-once-use-everywhere. The domain layer is domain-specific but structurally consistent. The task layer is about capabilities that can be composed.

This pattern isn't theoretical. Google's Knowledge Graph, Amazon's product ontology, and Uber's domain services all use layered ontological thinking. The shift now is bringing these ideas into the world of LLM-based agents, where the ontology doesn't just store knowledge — it **guides reasoning, constrains hallucination, and enables genuine domain understanding.**

---

*Before we go further, here's something worth sitting with:*

*LLMs are trained on billions of data points. They clearly learn something about how concepts relate — doctors treat patients, companies employ people, dependencies can conflict. Ask the right question and they'll traverse implicit relationships without any graph in sight.*

*So the uncomfortable question is: Do LLMs already have an implicit ontology baked into their weights? And if they do — why go through the trouble of building explicit ones?*

*I have thoughts. But I want yours first. Drop a comment with where you land — we'll unpack this properly in Part 2.*

---

## The Series Roadmap

Over the next nine posts, we'll build this up hands-on:

**Part 2** — Triples, Graphs, and Your First Knowledge Graph (with a companion Python script)

**Part 3** — Deep dive into ontological stratification: theory and design principles

**Parts 4–6** — Build each layer from the ground up: upper, domain, and task ontologies

**Part 7** — The practical semantic web stack: RDF, OWL, and what you actually need

**Part 8** — Full hands-on construction of a layered ontology

**Part 9** — Architecture patterns for knowledge-graph-powered agents

**Part 10** — Living ontologies: evolution, multi-agent KGs, and the road ahead

Each post uses pseudo-code for clarity, with companion Python scripts linked from a [GitHub repository](#) for the moments when running code teaches something that reading can't.

## A Taste: Flat vs. Structured Knowledge

To make the difference tangible, a companion Python script for this post `'01_flat_vs_structured.py'` in the series [GitHub repo](#) builds a tiny knowledge graph about people, organizations, and skills. It demonstrates how graph traversal answers questions that keyword search simply cannot.

The question: *"Who works at an organization that uses Python?"*

Keyword search needs "Python" to appear near a person's name. The knowledge graph follows edges — Person → worksAt → Organization → uses → Technology — and finds the answer through structure, not string matching.

**Structure is reasoning infrastructure.** That's the key insight of this entire series.

## References

1. [Gruber, T. \(1993\) — "A Translation Approach to Portable Ontology Specifications"](#)— The foundational definition of ontologies in AI

2. Guarino, N. (1998) — “Formal Ontology in Information Systems” — Ontological stratification and formal distinctions
3. W3C — OWL Web Ontology Language Overview— The web standard for ontologies
4. Hogan et al. (2021) — “Knowledge Graphs” — Comprehensive survey of KG theory and practice

Artificial Intelligence

Ontology

Agentic Ai

Knowledge Graph

Ai Knowledge Management



Follow

## Written by Sai

18 followers · 15 following

Curious generalist. Writing to learn — AI, philosophy, cognition, fitness. Sharing half-formed ideas to find the perspectives I'm missing.

## Responses (2)



Konrad "Stellars" Jelen

What are your thoughts?



Muhammad Awais

Mar 13



Really interesting perspective on how ontologies can act as the missing semantic layer for AI agents. The explanation of layered ontologies and knowledge graphs as a structural backbone for reasoning was especially insightful.

I explored a related... [more](#)



[Reply](#)



eejai42 he/him  
Mar 13



This is a fantastic primer on Ontologies. “Ontologies” will be the tech term of 2026 imho. This is their year!!



1 reply

[Reply](#)

## More from Sai

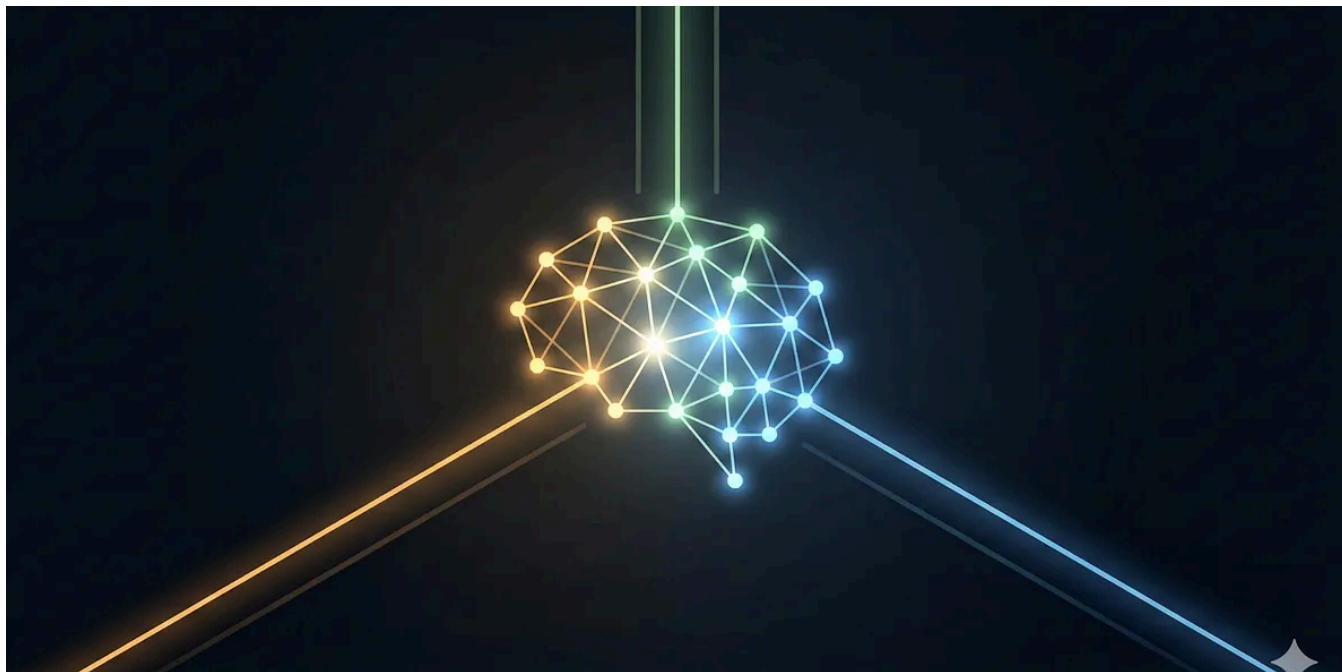


Sai

### Part 2 : Triples, Graphs, and Your First Knowledge Graph

The atomic unit of knowledge is three things connected. Everything else — traversal, reasoning, constraint checking — follows from that.

Mar 2 🖱 6

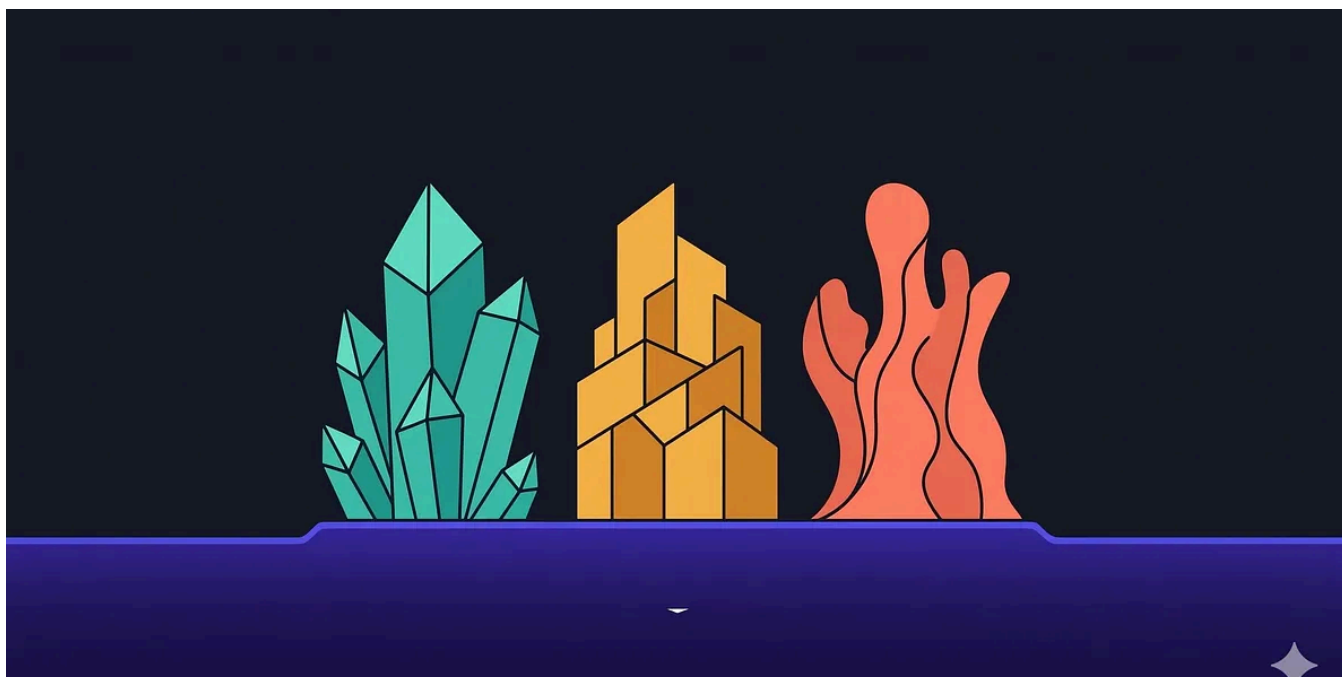


Sai

## From Schemas to Statistical Shadows—What Cognitive Science Tells Us About LLMs

Cognitive psychologists, AI researchers, and knowledge engineers—working independently—converged on the same structure for...

Feb 19 🖱 1



Sai

## Part 4 — The Upper Ontology: What All Your Agents Should Agree On

Three philosophical traditions. Thirty-six classes vs. twenty-five thousand terms. And a surprisingly practical question underneath it all...

Mar 16



In Building Ferri by Sai

## Your Phone Data Is Personal. Your AI Agent Should Be Too.

AI assistants are finally getting access to your phone. But who controls that access — you, or the company that built the assistant?

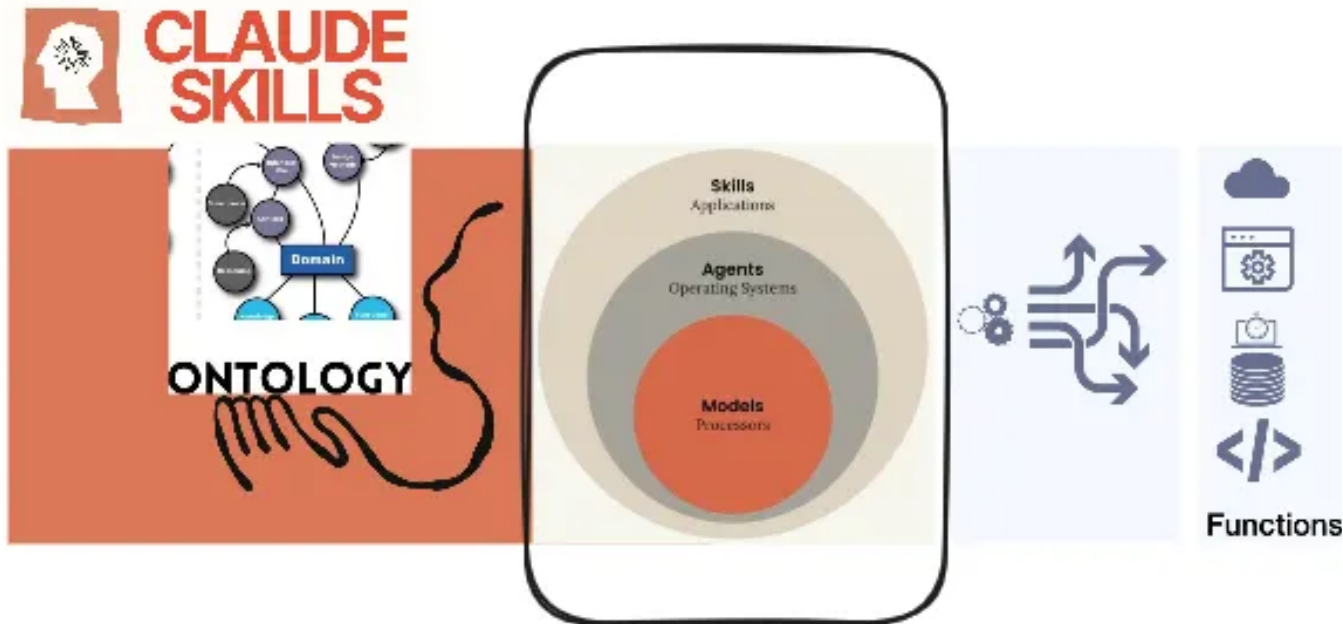
Feb 27



5

[See all from Sai](#)

## Recommended from Medium

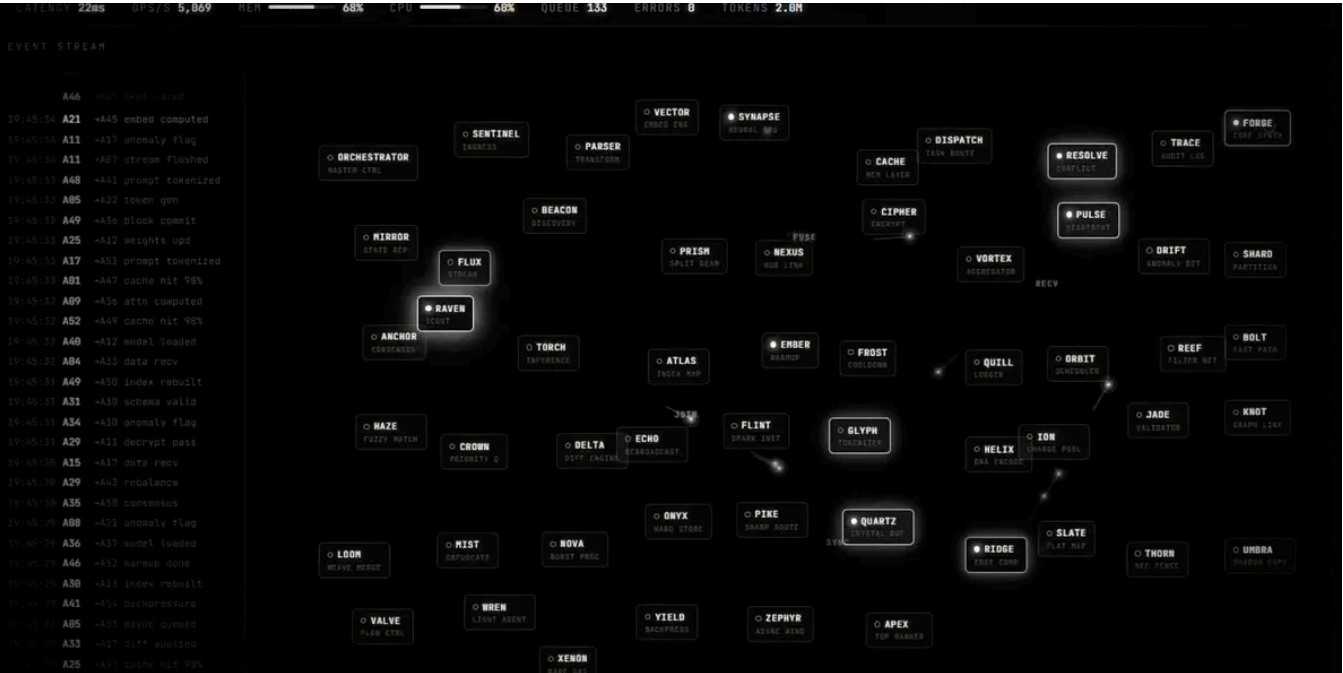


 Minyang Chen

## Enrich agents with domain-specific knowledge to make AI smarter (Ontology, Skills, RAG, Tool...

Finding new ways to expand an AI agent’s knowledge is a fascinating challenge. Last year I focused on using comprehensive “ontologies”...

★ Mar 10 🖱️ 4 💬 1  ...

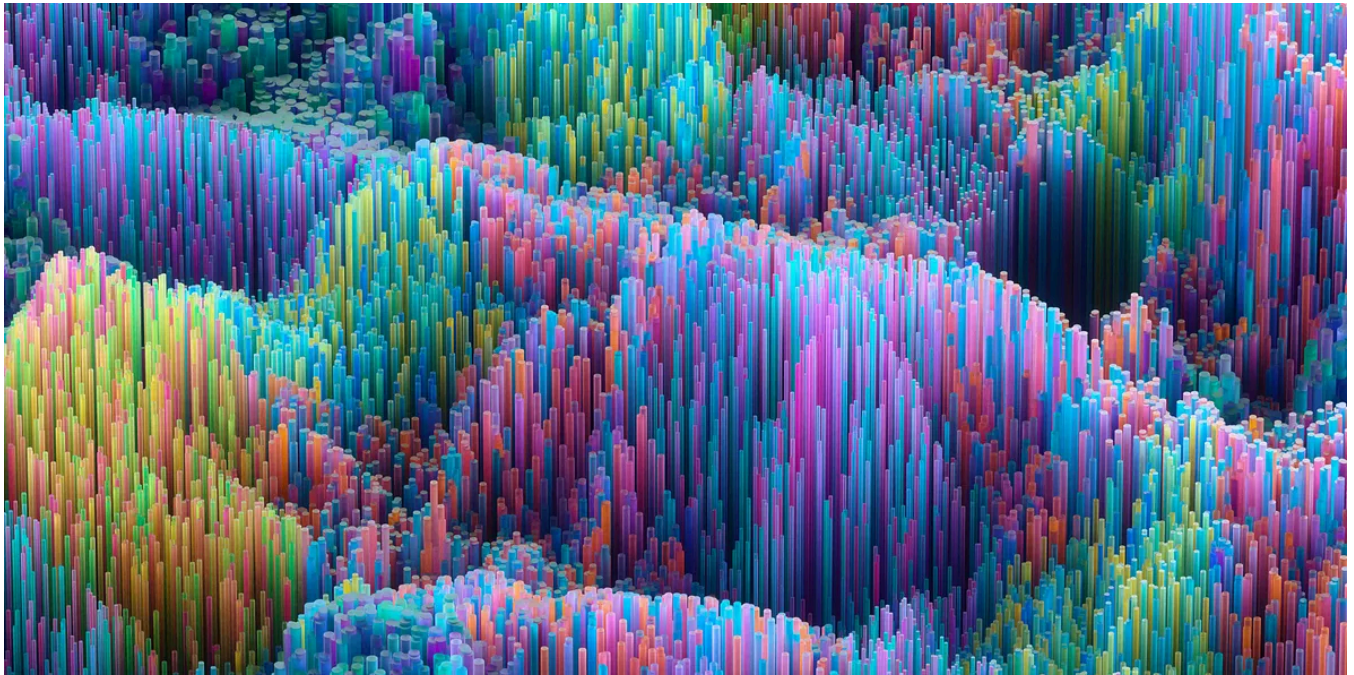


 Agent Native

## MiroFish: Swarm-Intelligence with 1M Agents That Can Predict Everything

Spawning thousands of autonomous agents with unique personalities, memories and social connections in simulated world to predict any event.

★ Mar 16 🖱 480 💬 12

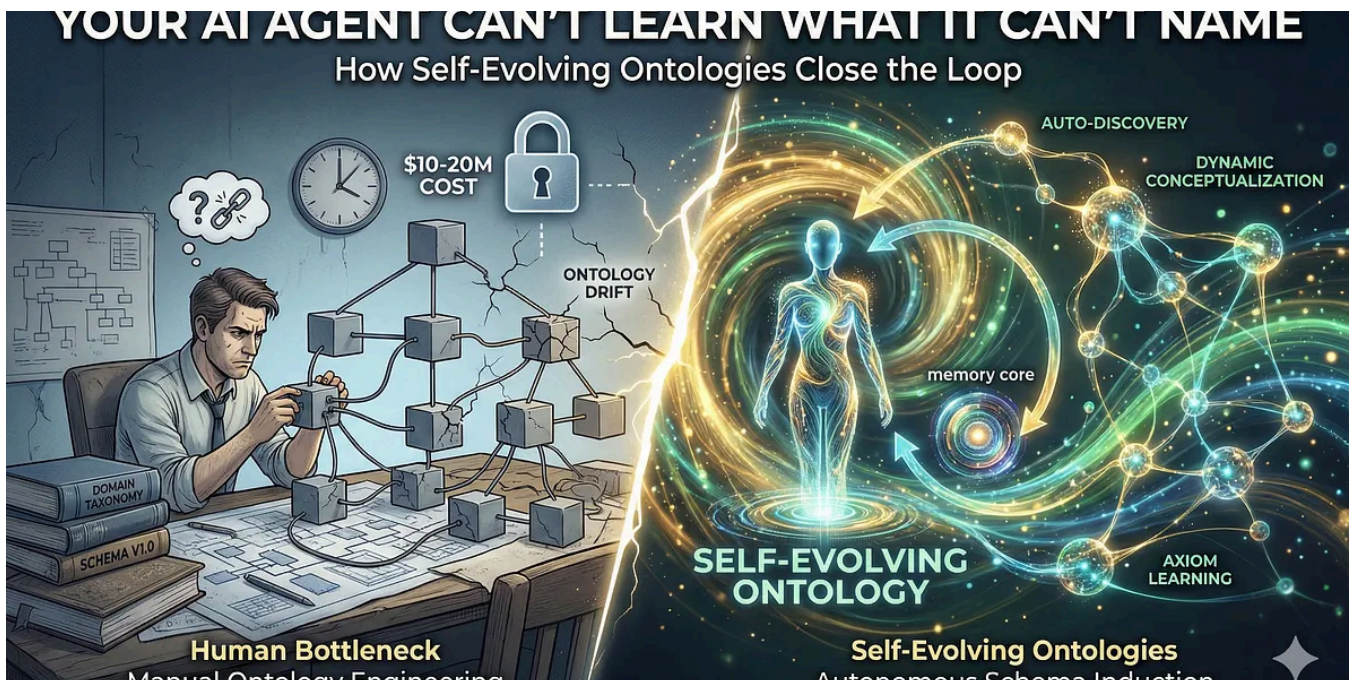


 David R Oliver

## Stop Drawing Diagrams — Generate Them

How five Claude Code skills turned architecture visualisation from a maintenance burden into a byproduct of good note-keeping.

★ Feb 28 🖱 164 💬 2



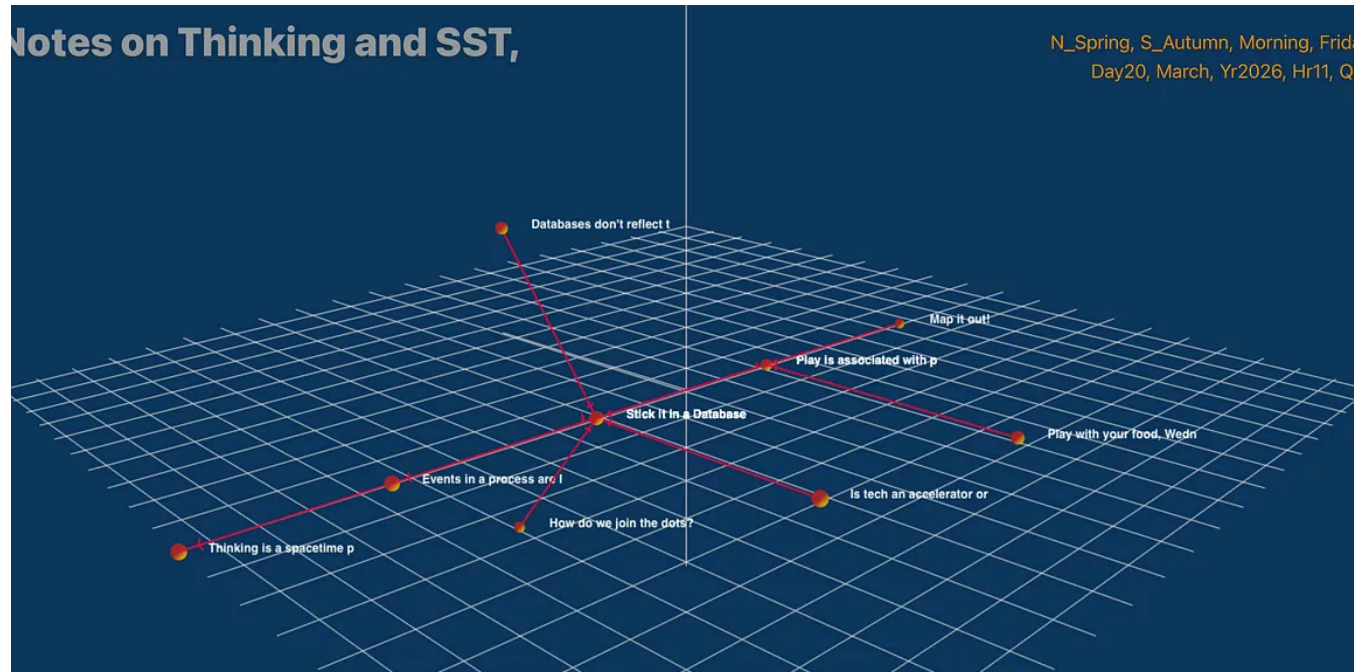


In Graph Praxis by Alexander Shereshevsky

## Your AI Agent Can't Learn What It Can't Name: How Self-Evolving Ontologies Close the Loop

The biggest bottleneck in knowledge graph adoption isn't the database, the model, or the data. It's the ontology — and the handful of...

★ Feb 15 🖱️ 4 💬 1



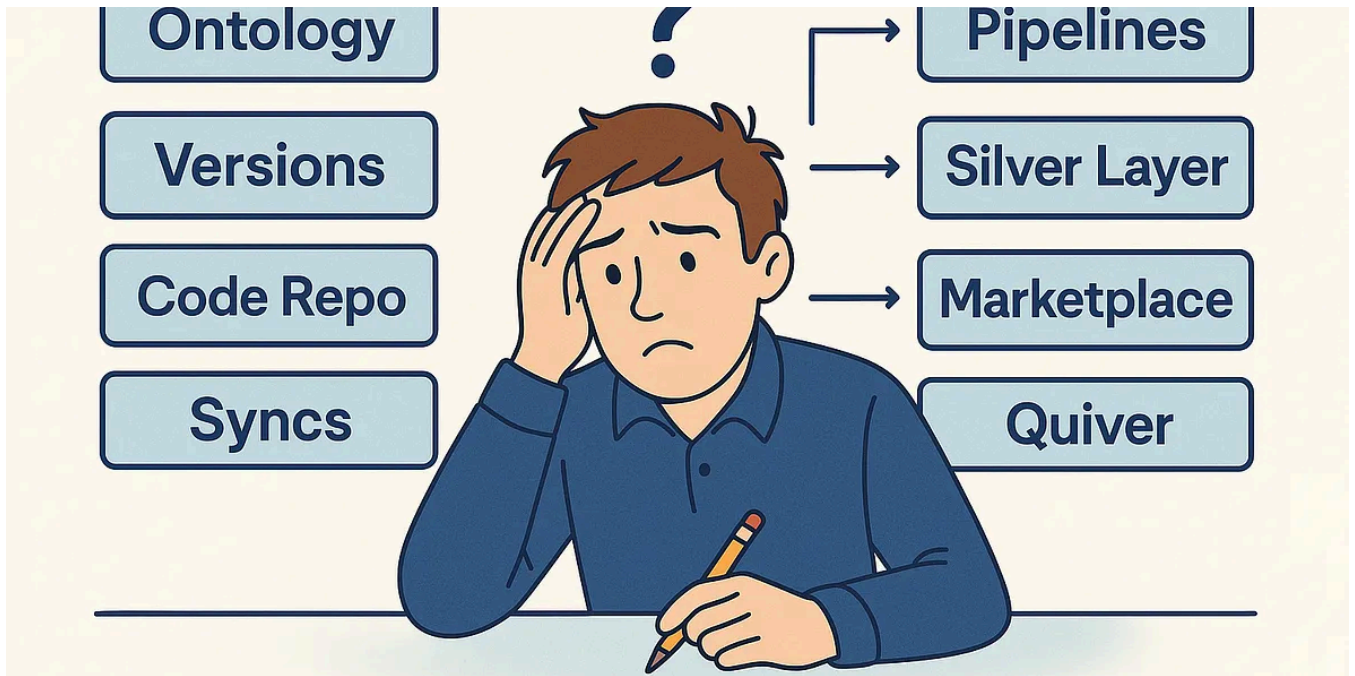
Mark Burgess

## Curating Your Own Thinking Experience, with Semantic Notes?

The Art of SSTorytelling your own stream of consciousness

5d ago 🖱️ 49 💬 3





S Sachin Tekchandani

## I'm a Data Engineer, What Is Palantir Foundry?

👉 Introduction

Oct 28, 2025 🖱 2



See more recommendations